

0.1 Môi trường và dữ liệu thực nghiệm

0.1.1 Môi trường phát triển

Hệ thống được xây dựng chủ yếu dựa trên ngôn ngữ lập trình Python, kết hợp cùng các thư viện xử lý toán học như Numpy và Pandas để xử lý ma trận vector nhúng. Việc truy xuất và sinh ngôn ngữ tự nhiên được thực hiện thông qua API của mô hình ngôn ngữ lớn Google Gemini 2.5 Flash, qua thư viện *google-genai*. Đối với tác vụ giải thuật đánh giá sự tương đồng, hệ thống sử dụng mô hình nhúng *dangvantuan/vietnamese-document-embedding* có chiều dài 768 chiều. Giai đoạn tái xếp hạng được đảm nhiệm bởi mô hình Cross-Encoder *AITeamVN/Vietnamese_Reranker*. Khác với các hệ thống phụ thuộc thư viện kín, logic tìm kiếm từ khoá (Custom BM25) được code tay tinh chỉnh ở mức toán học (TF, DF, IDF), tạo sự chủ động tùy biến cho đặc thù tiếng Việt.

0.1.2 Dữ liệu thực nghiệm

Nguồn dữ liệu gốc được khai thác từ toàn bộ tài liệu Sách Giáo Khoa Tin học lớp 10, 11 và 12, thuộc cả hai bộ sách Cánh Diều và Kết Nối Tri Thức. Qua quá trình tiền xử lý, văn bản được làm sạch và dán nhãn phân cấp từ chương, bài đến sâu nhất là tiểu mục. Tổng cộng, dữ liệu được phân rã thành 2348 khối dữ liệu (chunks) và có chứa thuộc tính level để biểu diễn độ sâu, phục vụ trực tiếp cho vòng phục hồi phân cấp (Hierarchical RAG).

0.2 Tổng quan kiến trúc toàn bộ hệ thống

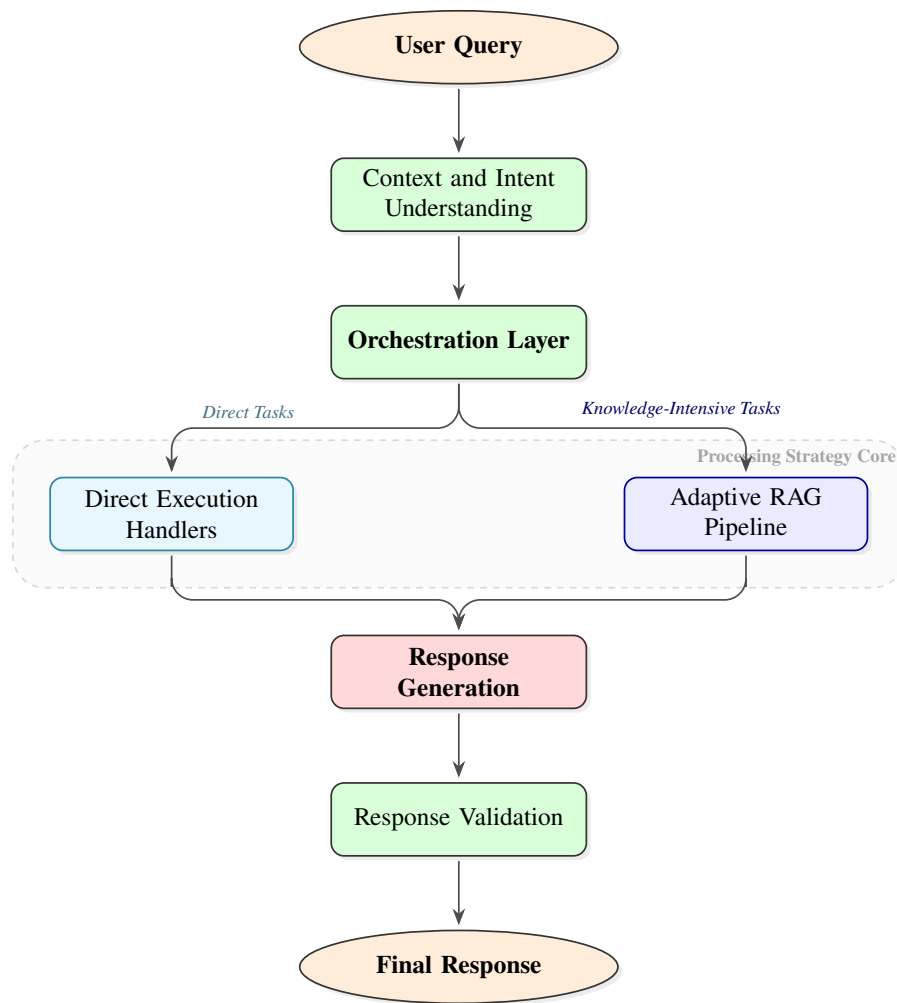
Kiến trúc tổng thể của dự án được thiết kế thành một pipeline (Data Pipeline) từ bước xử lý tài liệu thô ban đầu cho tới bước sinh câu trả lời hoàn thiện phục vụ người dùng. Quá trình này được chia làm các cụm thành phần nối tiếp nhau:

- **Cụm Tiền xử lý dữ liệu (Data Ingestion & OCR):** Nguồn dữ liệu dạng giáo trình (PDF) được nạp vào và đi qua mô hình khai phá OCR để nhận dạng, chuyển đổi văn bản sang định dạng Markdown theo chuẩn của LlamaParse. Việc này nhằm bóc tách cấu trúc phân cấp tinh vi (chương, bài, tiểu mục) của giáo trình.
- **Cụm Xây dựng Cơ sở tri thức (Knowledge Base Construction):** Văn bản Markdown sau quá trình làm sạch sẽ được phân rã (Chunking) theo đúng phân cấp. Mỗi khối dữ liệu (chunk) được nhúng (Embedding) thành ma trận vector và lưu trữ vĩnh viễn ở Cơ sở tri thức (Knowledge Base / Vector DB) cùng với metadata (*hệ cấp* và *ID cha con*).
- **Cụm Phân hệ Truy xuất (Retrieval & Reranking):** Ở chặng run-time, hệ thống đón nhận truy vấn (User Query) từ người dùng. Cụm RAG làm nhiệm vụ

vụ định tuyến hướng đi, tìm kiếm không gian vector, tìm kiếm từ khoá và lai ghép (Hybrid Search + Reranker) nhằm đánh giá và rút trích các khối tri thức chứa ngữ cảnh sát nhất.

- **Cụm Sinh ngôn ngữ (LLM Generation):** Nhận Top-K ngữ cảnh cô đặc ở đầu ra của cụm Truy xuất, kết hợp cùng Prompt (câu hỏi gốc) để truyền vào mô hình ngôn ngữ lớn (Google Gemini). LLM sẽ phân tích, đối chiếu nội dung ngữ cảnh từ tri thức và sinh ra câu trả lời theo ngôn ngữ tự nhiên.

Sơ đồ tổng quan được biểu diễn tại Hình 0.1, mô tả toàn bộ quy trình xử lý từ khi tiếp nhận truy vấn của người dùng cho đến khi sinh ra câu trả lời cuối cùng. Kiến trúc hệ thống theo mô hình tuổi thọ yêu cầu (request lifecycle) với 5 giai đoạn chính: Định tuyến ý định (Intent Routing), Phối hợp (Orchestration), Điểm chọn hướng (Dispatcher), Xử lý đôi đường (Dual-Path Processing), và Sinh phản hồi & Xác thực (Generation & Validation). Lỗi thông minh nhất của kiến trúc là **Dispatcher** chịu trách nhiệm quyết định định tuyến giữa hai đường xử lý: (i) đường trực tiếp cho các truy vấn đơn giản hoặc yêu cầu có tính chất chuyên biệt, và (ii) đường Adaptive RAG cho các truy vấn phức tạp cần lấy ngữ cảnh từ tri thức tích lũy.



Hình 0.1: Sơ đồ tổng quan kiến trúc hệ thống phiên bản nâng cấp thẩm mỹ (System Overview Architecture)

Trong Hình 0.1, kiến trúc hệ thống hoạt động theo các giai đoạn tuần tự và liên quan như sau:

Giai đoạn 1 - User Query & Intent Understanding: Bắt đầu từ truy vấn của người dùng, hệ thống thực hiện phân tích ngữ cảnh cuộc hội thoại và phát hiện ý định cốt lõi để xác định phạm vi xử lý cần thiết.

Giai đoạn 2 - Orchestration Layer: Đóng vai trò lớp phối hợp, quản lý trạng thái phiên làm việc và lập kế hoạch các hành động cụ thể dựa trên ý định đã phân tích.

Giai đoạn 3 - Dual-Path Processing Strategy: Tại đây, hệ thống phân nhánh thành hai con đường xử lý chuyên biệt gồm *Direct Execution Handlers* dành cho các yêu cầu đơn giản, lệnh trực tiếp không cần tra cứu tri thức và *Adaptive RAG Pipeline* kích hoạt quy trình truy xuất tri thức phân cấp từ kho dữ liệu SGK cho các câu hỏi phức tạp.

Giai đoạn 4 - Response Generation: Cả hai đường quy tụ lại tại mô hình ngôn

ngữ lớn để tổng hợp thông tin và sinh ra câu trả lời tự nhiên nhất theo đúng ngữ cảnh đã được cung cấp.

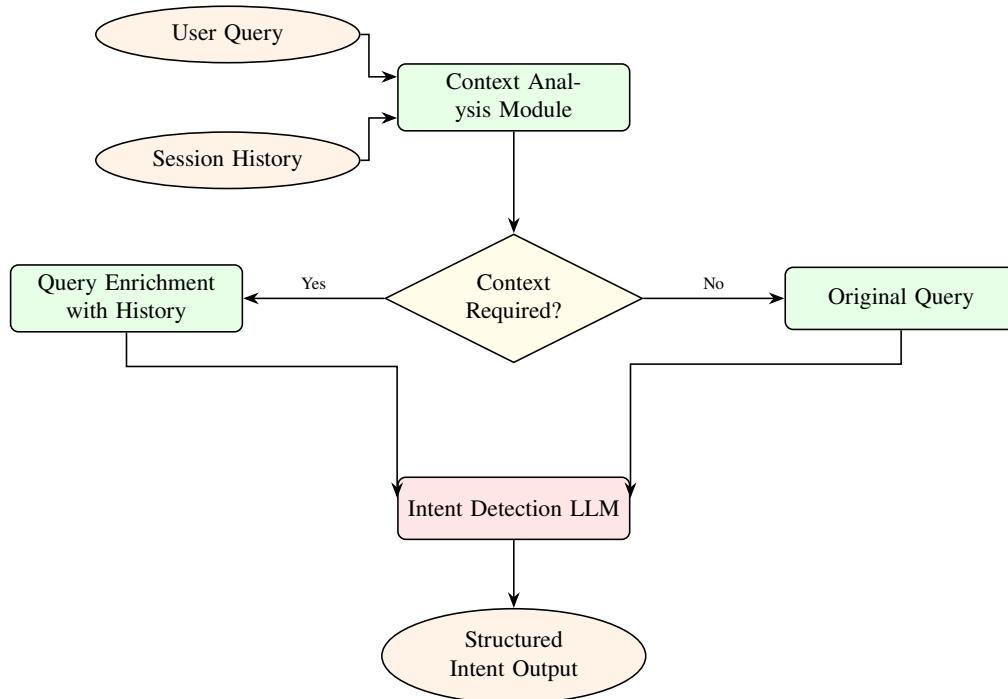
Giai đoạn 5 - Response Validation & Final Response: Kết quả sinh ra được đi qua một lớp kiểm tra (Validation) để đảm bảo tính chính xác, logic và không vi phạm các ràng buộc an toàn trước khi gửi phản hồi cuối cùng tới người dùng.

0.3 Thiết kế chi tiết các thành phần hệ thống

Dựa trên sơ đồ kiến trúc tổng quát tại Hình 0.1, các thành phần cốt lõi của hệ thống được thiết kế chi tiết nhằm đảm bảo tính linh hoạt và độ chính xác cao trong quá trình xử lý:

0.3.1 Thành phần Context and Intent Understanding

Đây là lớp tiếp nhận đầu tiên, thực hiện nhiệm vụ bóc tách ngữ cảnh (Context-Analyzer) và định danh ý định (IntentDetector). ContextAnalyzer phân tích lịch sử hội thoại để làm rõ các đại từ thay thế hoặc các thực thể bị ẩn ý, trong khi Intent-Detector sử dụng mô hình ngôn ngữ lớn để phân loại truy vấn vào các nhóm nhiệm vụ như: hỏi đáp tri thức, thực thi lệnh hệ thống, hoặc trò chuyện thông thường.

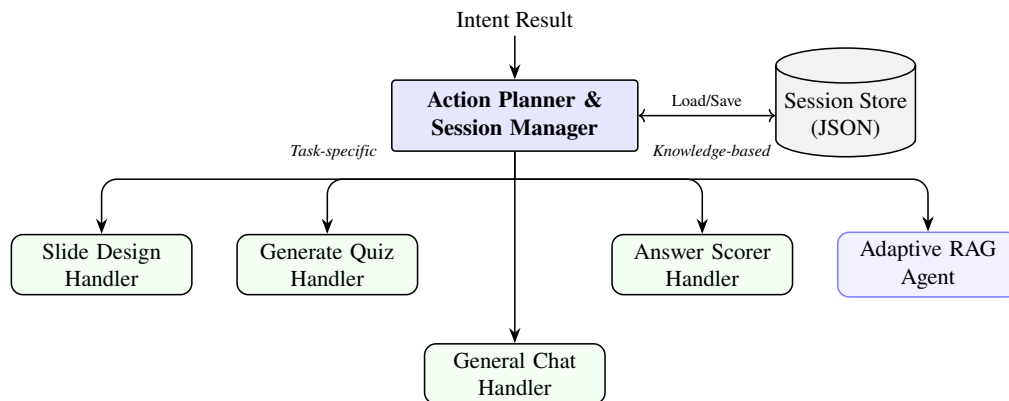


Hình 0.2: Chi tiết quy trình phân tích ngữ cảnh và nhận dạng ý định

Dữ liệu đầu vào của khối này là *User Query* thô cùng với *Conversation History* từ cơ sở dữ liệu phiên. Đầu ra là một cấu trúc dữ liệu *Intent Object* bao gồm nhãn ý định và các thực thể trích xuất, phục vụ trực tiếp cho việc lập kế hoạch tại tầng tiếp theo.

0.3.2 Thành phần Orchestration Layer

Sau khi có ý định rõ ràng, lớp phối hợp sẽ đảm nhiệm vai trò điều phối luồng công việc. Thành phần SessionManager chịu trách nhiệm duy trì trạng thái hội thoại, đảm bảo tính nhất quán của thông tin qua nhiều lượt tương tác. ActionPlanner sẽ dựa trên kết quả phân tích ý định để lập lộ trình thực thi, quyết định xem một yêu cầu cần được xử lý trực tiếp hay cần phải kích hoạt quy trình truy xuất tri thức hỗ trợ.

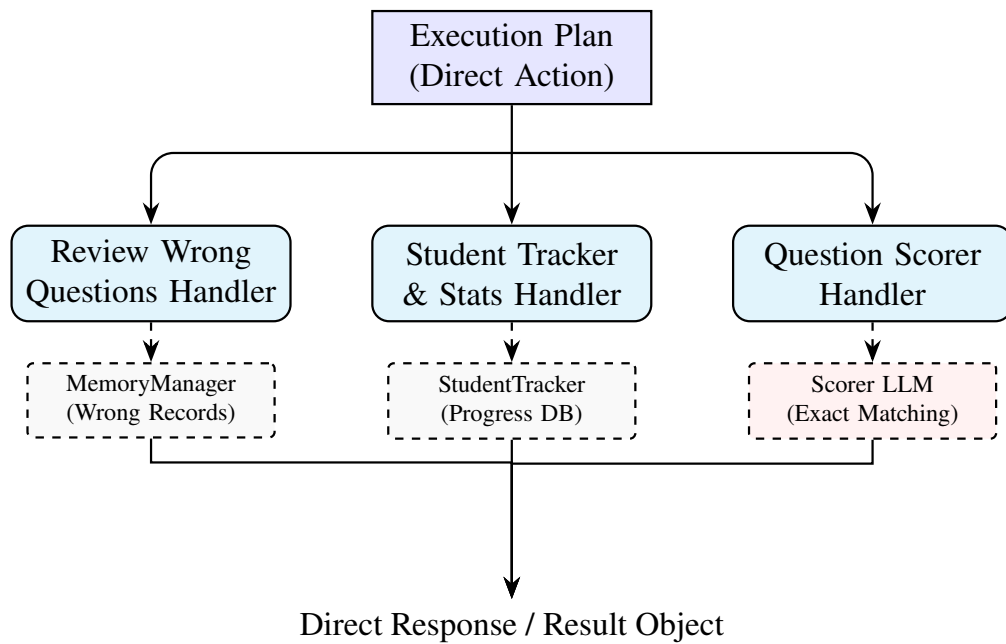


Hình 0.3: Kiến trúc điều phối (Orchestration) và cơ chế phân phối tác vụ (Task Dispatching)

Đầu vào của khối là *Intent Object* từ tầng trước. Đầu ra của Orchestration là một *Execution Plan* mô tả các bước cần thực hiện và định tuyến (routing) hướng đi cho Dispatcher. Trạng thái sau mỗi bước được *SessionManager* đồng bộ xuống *SessionStore* để phục vụ cho các lượt hội thoại kế tiếp.

0.3.3 Thành phần Direct Execution Handlers

Đối với các "Direct Tasks", hệ thống cung cấp các bộ xử lý chuyên biệt (Specialist Handlers). Các handler này được thiết kế để giải quyết nhanh chóng các tác vụ như: tra cứu định nghĩa ngắn, thực thi các hàm logic đã định nghĩa trước, hoặc các yêu cầu mang tính chất thủ tục mà không yêu cầu tìm kiếm ngữ cảnh rộng từ kho dữ liệu RAG.

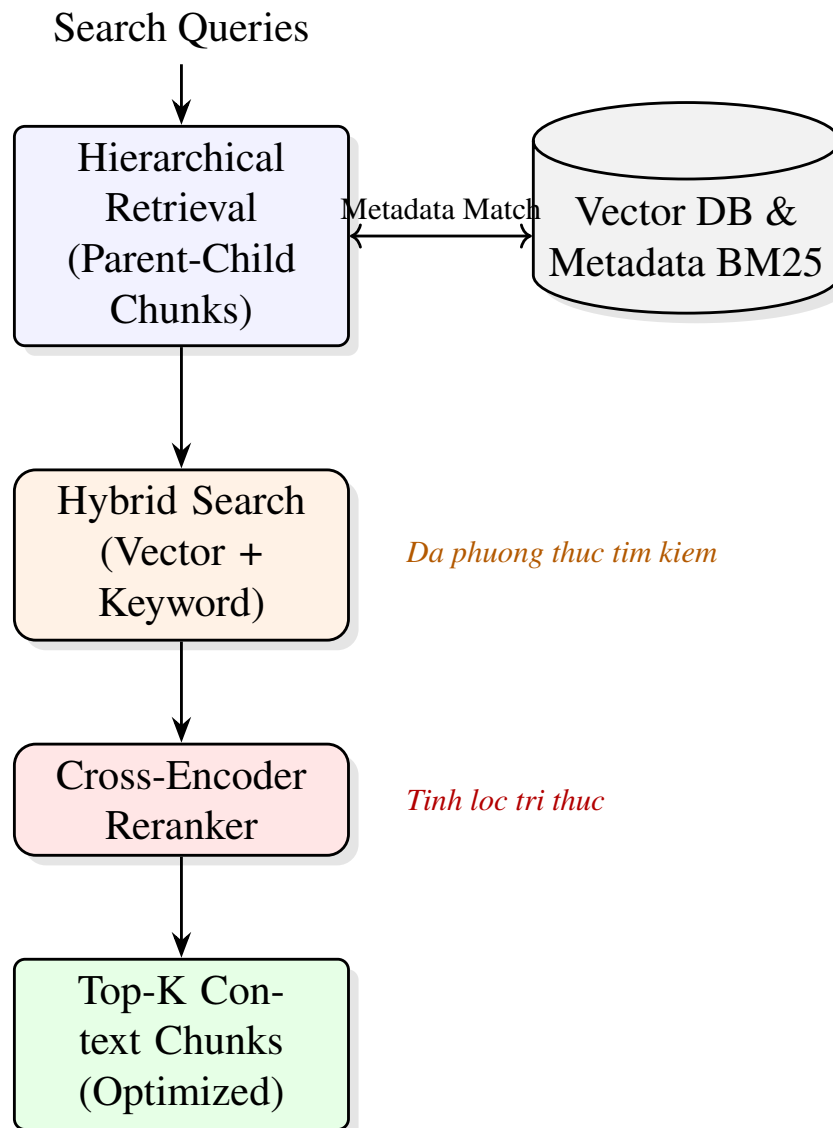


Hình 0.4: Sơ đồ luồng xử lý của các bộ thực thi trực tiếp (Direct Execution Handlers)

Khối này nhận *Execution Plan* và các tham số thực thi. Sau khi xử lý, nó trả về *Raw Result* trực tiếp cho khối sinh phản hồi mà không đi qua quy trình tìm kiếm vector phức tạp, giúp giảm thiểu độ trễ cho các yêu cầu đơn giản.

0.3.4 Thành phần Adaptive RAG Pipeline

Đây là thành phần chịu trách nhiệm cho các "Knowledge-Intensive Tasks". Pipeline này không hoạt động theo một luồng cố định mà có khả năng thích ứng (Adaptive) dựa trên độ khó của câu hỏi. Nó thực hiện quy trình truy xuất phân cấp (Hierarchical Retrieval) để lấy về các khối tri thức sát nhất với yêu cầu, sau đó sử dụng các thuật toán xếp hạng lai (Hybrid Search) để tinh lọc ngữ cảnh trước khi cung cấp cho mô hình sinh.



Hình 0.5: Chi tiết quy trình truy xuất tri thức thích ứng (Adaptive RAG Pipeline)

Đầu vào là các *Search Keywords* hoặc *Query Embeddings*. Đầu ra của pipeline là danh sách *Top-K Context Chunks* đã được tái xếp hạng (Reranked), đảm bảo chỉ những thông tin giá trị nhất mới được đưa vào mô hình sinh. Quy trình này bao gồm việc kết hợp giữa tìm kiếm vector (Dense Retrieval) và tìm kiếm từ khóa (Sparse Retrieval) thông qua thuật toán Reciprocal Rank Fusion (RRF), sau đó sử dụng mô hình Cross-Encoder để đánh giá lại mức độ liên quan thực tế giữa truy vấn và từng đoạn văn bản.

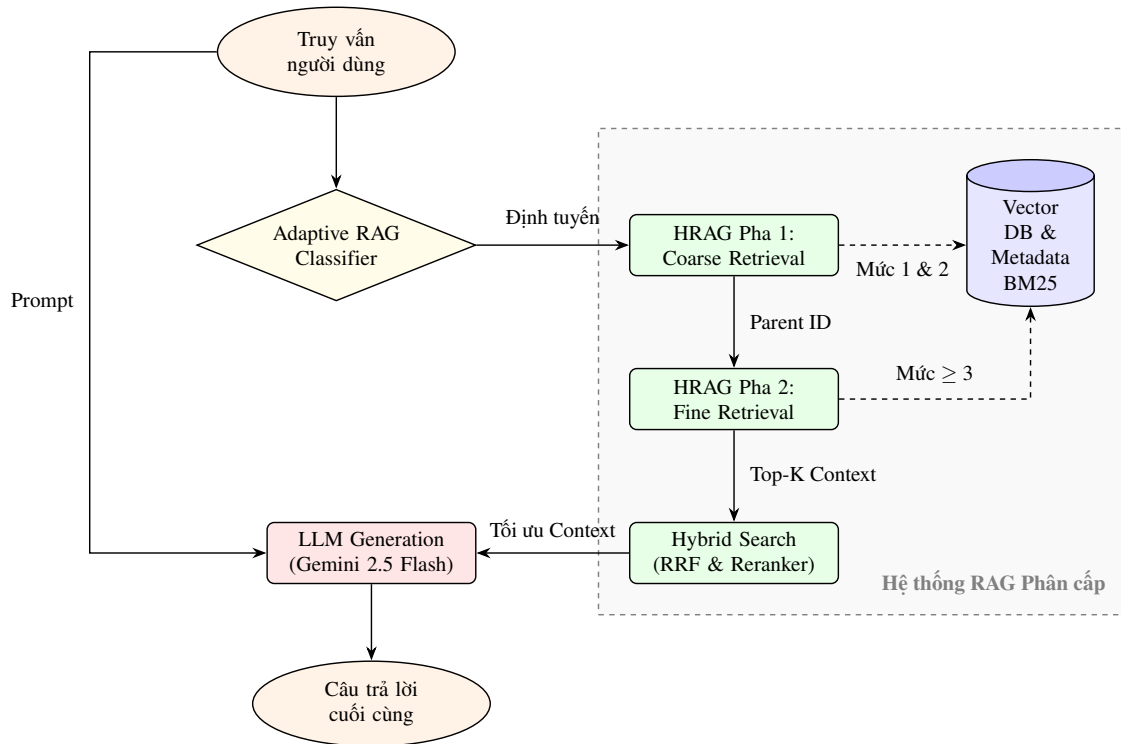
0.3.5 Thành phần Generation and Validation

Cuối cùng, Response Generation nhận ngữ cảnh tinh lọc để biên soạn câu trả lời. Điểm khác biệt quan trọng là sự xuất hiện của Validator Agent, một lớp hậu kiểm thực hiện đối soát câu trả lời vừa sinh ra với nguồn tri thức gốc để phát hiện các lỗi "hallucination" (ảo tưởng), đảm bảo rằng mọi thông tin gửi tới người dùng

đều có căn cứ xác thực từ sách giáo khoa.

Đầu vào của khối gồm *Context Chunks* và *Original User Prompt*. Đầu ra cuối cùng (*Final Response*) là văn bản ngôn ngữ tự nhiên đã qua kiểm định chất lượng, sẵn sàng hiển thị trên giao diện người dùng.

0.4 Thiết kế kiến trúc hệ thống RAG



Hình 0.6: Kiến trúc tổng quát hệ thống Adaptive - Hierarchical RAG

0.4.1 Tuỳ biến chiến lược Adaptive RAG

Kiến trúc hệ thống RAG hiện tại được nâng cấp lên cơ chế Adaptive RAG. Cơ chế này tiếp nhận truy vấn đầu vào và sử dụng *QueryClassifier* dưới dạng Agent để rút ra lộ trình phục hồi thích hợp. Thành phần Agent sẽ đánh giá bám sát và quyết định *RetrievalStrategy* trong 4 loại mặc định, trong đó quan trọng nhất là chiến lược tìm kiếm *Hierarchical*. Tùy theo kết quả đánh giá này mà bước tìm kiếm tiếp theo có sự thích ứng cụ thể và chính xác.

0.4.2 Kỹ thuật truy xuất phân cấp (Hierarchical RAG - HRAG)

Kỹ thuật *Hierarchical RAG* đóng vai trò cốt lõi nhằm tương thích hoàn toàn với dữ liệu có tính cấu trúc của SGK, qua đó khắc phục hiện tượng mất ngữ cảnh rộng khi chỉ dùng BM25 thông thường. Kỹ thuật này được chia thành hai giai đoạn độc lập và liên hiệp mật thiết: Pha 1 (Coarse Retrieval) và Pha 2 (Fine Retrieval).

Trước hết, trong Pha 1, hệ thống thực hiện tìm kiếm ngữ nghĩa thuần túy dưới dạng mô hình vector đối với đại diện khối gốc (lớp cha). Các khối này là các bản

ghi chứa tên chương, bài và không chứa nội dung chi tiết nằm ở mức *level 1* và *level 2*. Những vector chunk có điểm số Semantic lớn hơn một ngưỡng cố định (khoảng 0.55) sẽ được trả về, cho biết *mã khóa* parent tiềm năng nhất.

Tiếp đến, ở Pha 2 (Fine Retrieval), mã khóa của level cha lấy được sẽ đóng vai trò khoanh vùng. Mô hình tìm kiếm thứ hai - tìm kiếm lai (Hybrid Search) - sẽ hoạt động chỉ trên các khối dữ liệu con chứa nội dung chi tiết ($level \geq 3$) mà thuộc quyền sở hữu của danh sách *parent* kể trên. Kỹ thuật này giúp loại bỏ nhiều nhiễu so với tìm kiếm nhắm thẳng vào toàn bộ dữ liệu, giảm chi phí tính toán nhờ thu hẹp vùng nháp, và đảm bảo ngữ cảnh tài liệu tập trung.

0.4.3 Tìm kiếm lai (Hybrid) và sắp xếp (RRF, Reranker)

Không dừng lại ở Semantic, kết quả được củng cố cùng Custom BM25 thuật toán thông qua hàm lai tiêu chuẩn là Reciprocal Rank Fusion (RRF). RRF liên kết thứ hạng của tìm kiếm chính xác từ khóa với tìm kiếm vector trình độ tương tự, thiết lập tham số $\omega = 60$. Cuối cùng, tập kết quả RRF sẽ được tái xếp hạng lại bằng Cross-Encoder Reranker, dựa trên trọng số logits giữa câu hỏi và từng chunk, giúp lọc ra *Top-K* chính xác nhất để tối ưu hóa việc truyền vào mô hình LLM chuyên trách sinh câu trả lời (Generation).

0.5 Đánh giá và kết quả đạt được

Quá trình đánh giá hiệu suất được đo lường nghiêm ngặt bằng hình thức thử nghiệm tự động từ framework RAGAS. Quy trình sử dụng bộ tham chiếu tự sinh ngẫu nhiên hoá gồm 246 mẫu câu hỏi tiêu chuẩn, bám sát SGK.

Kết quả thực tế cho thấy sáng kiến Adaptive RAG kết hợp HRAG tương tác tốt và kéo lên rõ rệt chỉ số *Faithfulness* (Độ tin cậy), đạt được điểm số trung bình là 0.9757. Thông số này tương đương với mức độ ảo giác sinh ra từ LLM là cực thấp so với ngữ cảnh nạp vào. Đồng thời, biến số *Context Recall* đạt hiệu năng là 0.9593, chứng minh việc sử dụng HRAG phòng chống hiệu quả tình trạng sót thông tin khi tìm kiếm, khắc phục độ trôi ngữ cảnh khi tài liệu bị trộn lẫn. Trừ các mặt tổn kém thời gian đi tìm hai chặng trong HRAG, chiều thức này cho ra sự ưu việt đáng kể vào khả năng cá nhân hóa truy xuất trên tài liệu dạng cấu trúc giáo trình như SGK.